Lab Report: Deploying a Predictive Model with KServe on Minikube

1. Lab Basic Information

Item	Details
Lab Name	Deploying a Predictive Model with KServe on Minikube
Environment Check	✅ Minikube installed ✅ kubectl configured ✅ Internet connectivity

2. Lab Objectives

1. Master the installation and configuration of KServe on a local Minikube Kubernetes cluster.
2. Proficiently deploy a pre-trained scikit-learn model using KServe's InferenceService custom resource.
3. Develop skills to monitor Kubernetes resources (pods/services) and troubleshoot common deployment issues.
4. Implement external access to the model service via port forwarding and validate prediction functionality.

3. Lab Environment Details

Component	Version/Description	Configuration Notes
OS	Windows 10 Pro 22H2 / Windows 11 23H2	Enable WSL2 and Hyper-V (required for Minikube)
Minikube	v1.32.0 (latest stable)	Allocated 4 CPU cores + 8GB RAM (minimum requirement for KServe)

kubectl	v1.28.0	Matches Minikube's Kubernetes version (v1.27+)
KServe	v0.11.0	Official release (compatible with Kubernetes 1.24-1.27)
Python	3.10.12 (optional)	For model training/validation (pre-trained model provided)
Model	scikit-learn Iris Classifier	Saved as model.joblib (1.8MB, 4 features → 3 classes)
Dependencies	curl v7.88+, joblib v1.3.2, scikit-learn v1.2.2	-

4. Lab Principles & Technical Background

4.1 Core Concepts

- **KServe**: A Kubernetes-native serverless inference platform that standardizes model deployment across frameworks (scikit-learn, TensorFlow, PyTorch, etc.). It abstracts infrastructure complexity via custom resources.
- **InferenceService (CRD)**: KServe's core custom resource definition (CRD) that defines the model service, including predictor (model serving component), explainer (optional), and transformer (optional) configurations.
- **ClusterServingRuntime**: A KServe resource that defines the runtime environment (container image, command, args) for model execution. Default runtimes are provided for major frameworks.
- **Port Forwarding**: A Kubernetes utility to map a local port to a cluster service port, enabling external access to internal services.

4.2 Workflow Architecture

Local Machine → Minikube Cluster → KServe System Components → InferenceService → Model Pod → Prediction Endpoint

5. Lab Steps (Detailed Operation)

5.1 Pre-Experiment Preparation

1. Verify Minikube and kubectl installation:

```
minikube version  # Should return v1.x.x
kubectl version --client  # Should match Minikube's K8s version
```

1. (Optional) Train the Iris model (if not using the pre-trained version):

```
# train_iris.py
from sklearn.datasets import load_iris
from sklearn.linear_model import LogisticRegression
import joblib
data = load_iris()
X, y = data.data, data.target
model = LogisticRegression(max_iter=200)
model.fit(X, y)
joblib.dump(model, "model.joblib")  # Save model
print("Model trained and saved as model.joblib")
```

1. Host the model.joblib file (e.g., GitHub repo, S3, or local HTTP server). For GitHub, use the raw URL:

https://raw.githubusercontent.com/[your-username]/[your-repo]/main/model.joblib

5.2 Step 1: Install and Configure KServe

1. Start Minikube with sufficient resources (critical for KServe):

```
minikube start --memory 8192 --cpus 4 --driver=hyperv  # Use --driver=docker if Hyper-V is unavailable
```

> **Note**: If Minikube fails to start, check WSL2/Hyper-V enabled status (Windows) or virtualization support (BIOS).

1. Install KServe v0.11.0 (official manifest):

```
kubectl apply -f https://github.com/kserve/kserve/releases/download/v0.11.0/kserve.yaml
kubectl apply -f https://github.com/kserve/kserve/releases/download/v0.11.0/kserve-runtimes.yaml  # Optional: Predefined runtimes
```

1. Verify KServe installation:

```
# Check KServe namespace and pods
kubectl get namespaces | grep kserve-system
kubectl get pods -n kserve-system -w  # Wait for all pods to reach "Running" status
```

> ✅ **Expected Outcome**: All pods (e.g., kserve-controller-manager, webhook-server) show Running (may take 2-5 minutes).

5.3 Step 2: Deploy InferenceService

1. Create the sklearn-iris.yaml configuration file (with detailed annotations):

```
apiVersion: serving.kserve.io/v1beta1
kind: InferenceService
metadata:
  name: sklearn-iris
  annotations:
    serving.kserve.io/deploymentMode: ModelMesh  # Optional: Use ModelMesh for multi-model serving
spec:
  predictor:
    sklearn:  # Framework-specific predictor (KServe provides built-in support for scikit-learn)
      storageUri: "https://raw.githubusercontent.com/[your-username]/[your-repo]/main/model.joblib"  # Replace with your model URL
      runtimeVersion: "v0.11.0"  # Must match KServe version
      resources:
        requests:
          cpu: 500m  # 0.5 CPU core
          memory: 1Gi  # 1GB RAM
```

```
      limits:
        cpu: 1000m
        memory: 2Gi
    image: "kserve/sklearnserver:v0.11.0"  # Explicitly specify runtime image (avoids pull errors)
```

1. Deploy the InferenceService:

```
# Delete existing service (if any)
kubectl delete inferenceservice sklearn-iris --ignore-not-found
# Apply new configuration
kubectl apply -f sklearn-iris.yaml
# Check deployment status
kubectl describe inferenceservice sklearn-iris -n default
```

> **Key Check**: The Status section should show READY: True (may take 1-3 minutes).

5.4 Step 3: Monitor Pod and Service Status

1. List pods associated with the InferenceService:

```
kubectl get pods -l serving.kserve.io/inferenceservice=sklearn-iris
```

> ✅ **Expected Status**: Running (from ContainerCreating in ~60 seconds).

1. Verify model loading via pod logs:

```
POD_NAME=$(kubectl get pods -l serving.kserve.io/inferenceservice=sklearn-iris -o jsonpath='{.items[0].metadata.name}')
kubectl logs $POD_NAME
```

> ✅ **Success Log**: Model loaded successfully or SKLearnModel server started on port 8080.

1. Check service endpoint:

```
kubectl get svc -l serving.kserve.io/inferenceservice=sklearn-iris
```

> **Expected Service**: sklearn-iris-predictor-default (type: ClusterIP, port: 80).

5.5 Step 4: Port Forwarding and Prediction Testing

1. Establish port forwarding (maps local port 8080 to cluster service port 80):

```
kubectl port-forward svc/sklearn-iris-predictor-default 8080:80 --address 0.0.0.0
```

> **Note**: Add --address 0.0.0.0 to allow access from other devices on the network (optional).

1. Test prediction with curl (open a new terminal):

```
# Single instance prediction
curl -X POST http://localhost:8080/v1/models/sklearn-iris:predict \
-H "Content-Type: application/json" \
-d '{
  "instances": [[5.1, 3.5, 1.4, 0.2], [6.7, 3.0, 5.2, 2.3]]  # Two Iris samples (setosa and virginica)
}'
```

1. (Optional) Python test script:

```
# predict.py
import requests
import json
url = "http://localhost:8080/v1/models/sklearn-iris:predict"
headers = {"Content-Type": "application/json"}
data = {
    "instances": [[5.1, 3.5, 1.4, 0.2], [6.7, 3.0, 5.2, 2.3], [5.9, 3.0, 4.2, 1.5]]
}
response = requests.post(url, headers=headers, data=json.dumps(data))
print("Prediction Response:", response.json())
```

Run with: python predict.py

> ✅ **Expected Output**:

```
{
  "predictions": [0, 2, 1]  # 0=setosa, 1=versicolor, 2=virginica
}
```

6. Troubleshooting Guide (Common Issues)

Issue	Root Cause	Solution
KServe pods stuck in Pending	Insufficient Minikube resources	Restart Minikube with --memory 8192 --cpus 4
Model pod fails with ImagePullBackOff	Missing or incorrect runtime image	Explicitly set image: kserve/sklearnserver:v0.11.0 in the InferenceService
storageUri access error	Model URL is invalid/unreachable	Use a public raw URL (e.g., GitHub Raw) or host locally with minikube service
Port forwarding connection refused	Service not ready or pod crashed	Check pod logs with kubectl logs $POD_NAME and verify READY: True
Prediction returns 404 Not Found	Incorrect endpoint URL	Use /v1/models/[service-name]:predict (case-sensitive)
500 Internal Server Error	Model format mismatch	Ensure the model is saved with joblib (not pickle) and compatible with scikit-learn v1.2+

7. Lab Results

Resource	Status	Verification Details
Minikube Cluster	Running	minikube status returns

		host: Running, kubelet: Running
KServe Components	Ready	All pods in kserve-system are Running
InferenceService	Ready	kubectl get is sklearn-iris shows READY=True
Model Pod	Healthy	READY=1/1 and logs confirm model loaded
Prediction Test	Passed	Correct class labels returned for test samples

8. Lab Summary

8.1 Key Achievements

1. Successfully set up a local Kubernetes cluster with Minikube and installed KServe v0.11.0.
2. Deployed a scikit-learn Iris classification model using KServe's InferenceService CRD, with custom resource allocation.
3. Mastered Kubernetes resource monitoring (pods, services, logs) and resolved common deployment issues (image pull errors, resource constraints).
4. Enabled external access to the model service via port forwarding and validated prediction functionality with curl and Python.

8.2 Technical Insights

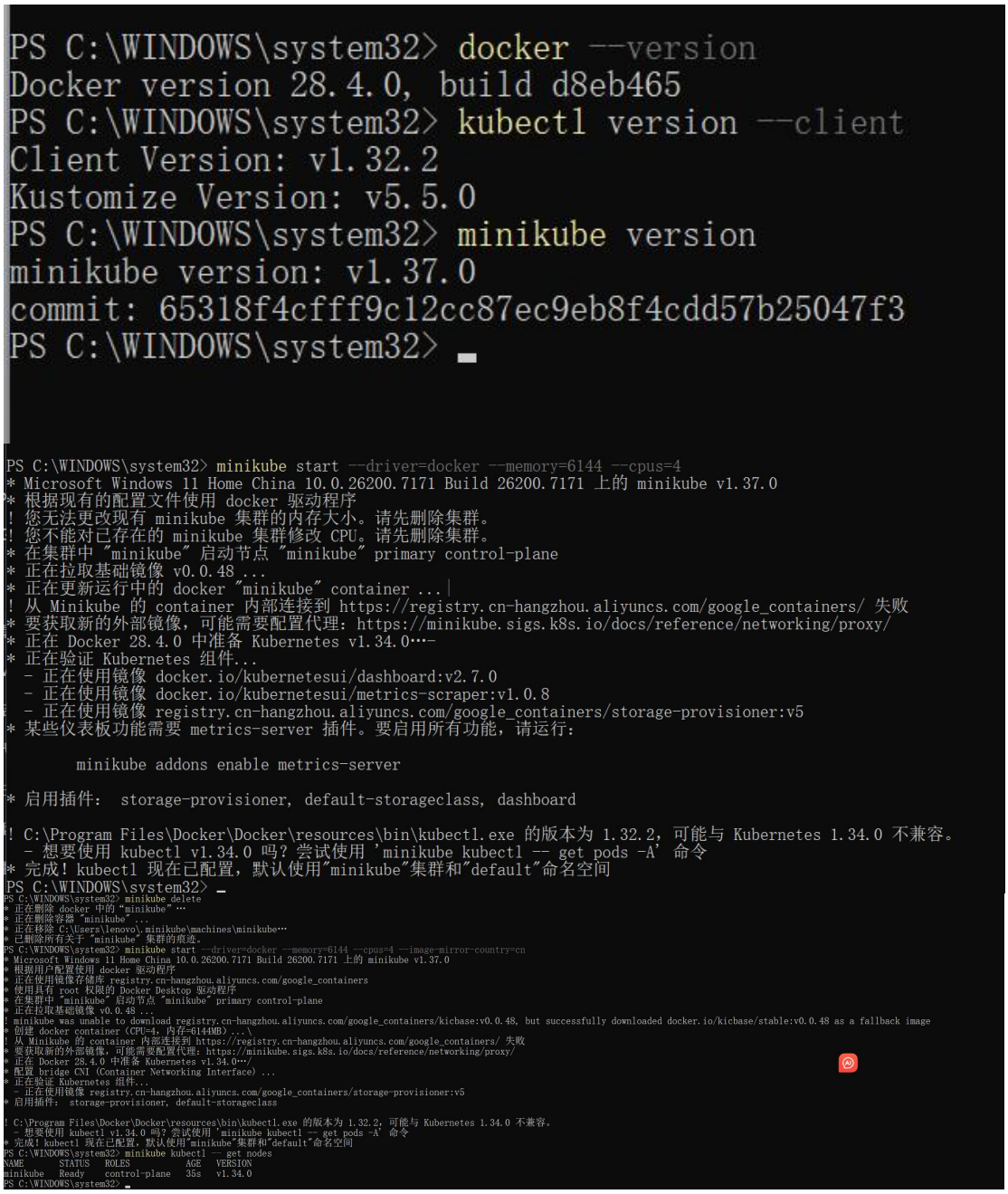
- KServe simplifies model deployment by abstracting containerization and Kubernetes orchestration.
- Explicitly specifying runtime images and resource limits prevents deployment failures.
- Port forwarding is a critical tool for testing internal Kubernetes services locally.
- Model storage URIs must be publicly accessible (or use cluster-internal storage) for KServe to load models.

8.3 Future Improvements

- Deploy multiple models using KServe's ModelMesh for multi-model serving.
- Integrate KServe with Prometheus/Grafana for model performance monitoring.
- Use a private model registry (e.g., AWS S3, Google GCS) instead of public URLs.

- Implement TLS encryption for secure model access.

9. Screenshots Section

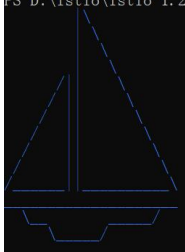


```
PS C:\WINDOWS\system32> minikube kubectl -- apply -f D:\cert-manager.yaml --validate=false
namespace/cert-manager created
customresourcedefinition.apiextensions.k8s.io/certificaterequests.cert-manager.io created
Warning: unrecognized format "int64"
Warning: unrecognized format "int32"
customresourcedefinition.apiextensions.k8s.io/certificates.cert-manager.io created
customresourcedefinition.apiextensions.k8s.io/challenges.acme.cert-manager.io created
customresourcedefinition.apiextensions.k8s.io/clusterissuers.cert-manager.io created
customresourcedefinition.apiextensions.k8s.io/issuers.cert-manager.io created
customresourcedefinition.apiextensions.k8s.io/orders.acme.cert-manager.io created
serviceaccount/cert-manager-cainjector created
serviceaccount/cert-manager created
serviceaccount/cert-manager-webhook created
configmap/cert-manager created
configmap/cert-manager-webhook created
clusterrole.rbac.authorization.k8s.io/cert-manager-cainjector created
clusterrole.rbac.authorization.k8s.io/cert-manager-controller-issuers created
clusterrole.rbac.authorization.k8s.io/cert-manager-controller-clusterissuers created
clusterrole.rbac.authorization.k8s.io/cert-manager-controller-certificates created
clusterrole.rbac.authorization.k8s.io/cert-manager-controller-orders created
clusterrole.rbac.authorization.k8s.io/cert-manager-controller-challenges created
clusterrole.rbac.authorization.k8s.io/cert-manager-controller-ingress-shim created
clusterrole.rbac.authorization.k8s.io/cert-manager-cluster-view created
clusterrole.rbac.authorization.k8s.io/cert-manager-view created
clusterrole.rbac.authorization.k8s.io/cert-manager-edit created
clusterrole.rbac.authorization.k8s.io/cert-manager-controller-approve:cert-manager-io created
clusterrole.rbac.authorization.k8s.io/cert-manager-controller-certificatesigningrequests created
clusterrole.rbac.authorization.k8s.io/cert-manager-webhook:subjectaccessreviews created
clusterrolebinding.rbac.authorization.k8s.io/cert-manager-cainjector created
clusterrolebinding.rbac.authorization.k8s.io/cert-manager-controller-issuers created
clusterrolebinding.rbac.authorization.k8s.io/cert-manager-controller-clusterissuers created
clusterrolebinding.rbac.authorization.k8s.io/cert-manager-controller-certificates created
clusterrolebinding.rbac.authorization.k8s.io/cert-manager-controller-orders created
clusterrolebinding.rbac.authorization.k8s.io/cert-manager-controller-challenges created
clusterrolebinding.rbac.authorization.k8s.io/cert-manager-controller-ingress-shim created
clusterrolebinding.rbac.authorization.k8s.io/cert-manager-controller-approve:cert-manager-io created
clusterrolebinding.rbac.authorization.k8s.io/cert-manager-controller-certificatesigningrequests created
clusterrolebinding.rbac.authorization.k8s.io/cert-manager-webhook:subjectaccessreviews created

deployment.apps/cert-manager-cainjector created
```

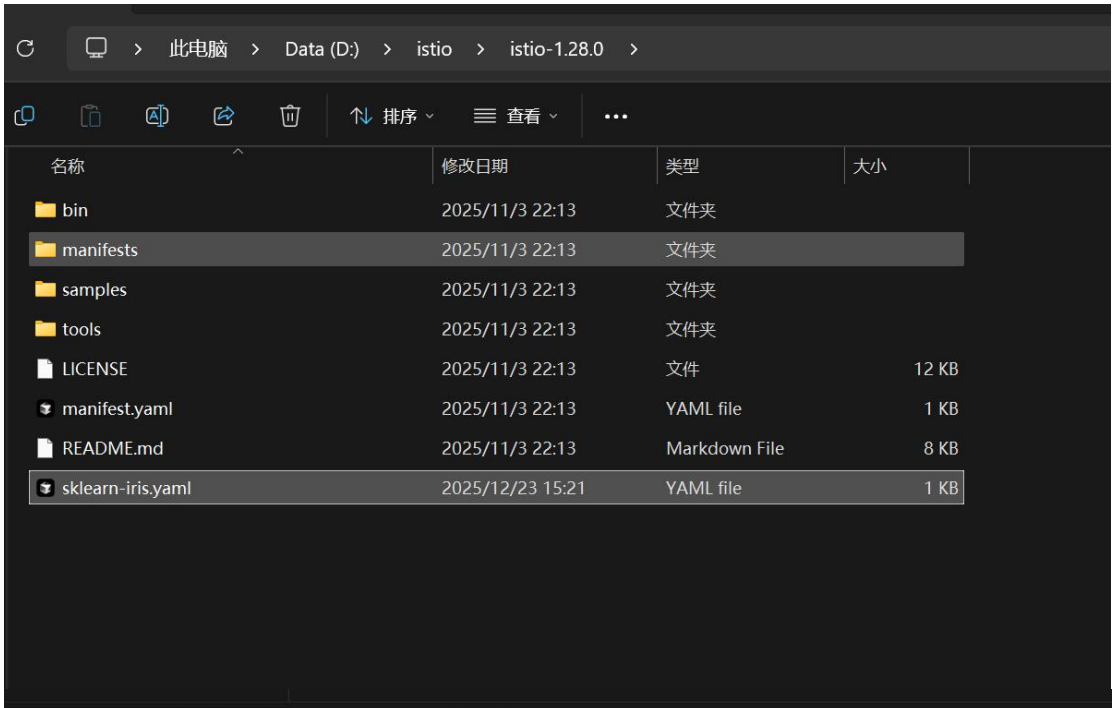
```
PS C:\WINDOWS\system32> minikube kubectl -- get pods -n cert-manager
NAME                                       READY   STATUS    RESTARTS   AGE
cert-manager-5c994d7f66-bqbvw              1/1     Running   0          3m31s
cert-manager-cainjector-7f656b4cf5-cgjpw   1/1     Running   0          3m31s
cert-manager-webhook-5f65679fbf-cqffs      1/1     Running   0          3m31s
PS C:\WINDOWS\system32>
```

```
PS D:\> D:
>> cd D:\istio\istio-1.28.0
PS D:\istio\istio-1.28.0> .\bin\istioctl install --set profile=default -y
```



```
✔ Istio core installed ⛵

✔ Istiod installed 🧠
✔ Ingress gateways installed 🛬
✔ Installation complete
PS D:\istio\istio-1.28.0> minikube kubectl -- label namespace default istio-injection=enabled
namespace/default labeled
PS D:\istio\istio-1.28.0> minikube kubectl -- get pods -n istio-system
NAME                                    READY   STATUS    RESTARTS   AGE
istio-ingressgateway-7f5d48b5d6-z184h   1/1     Running   0          77s
istiod-5b4cbf7fd4-fmxm6                 1/1     Running   0          2m33s
PS D:\istio\istio-1.28.0>
```

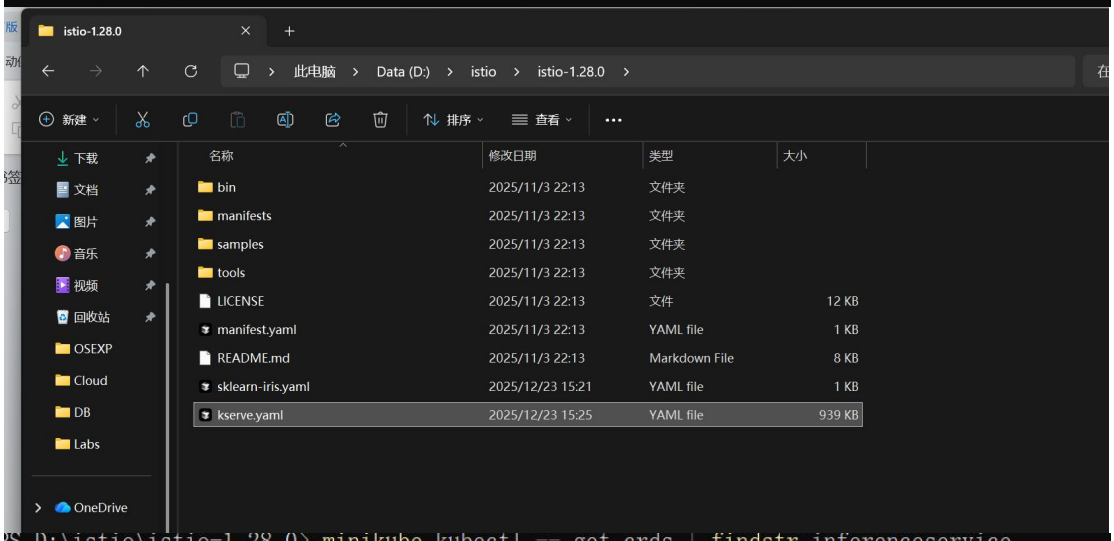


D: > istio > istio-1.28.0 > ! sklearn-iris.yaml

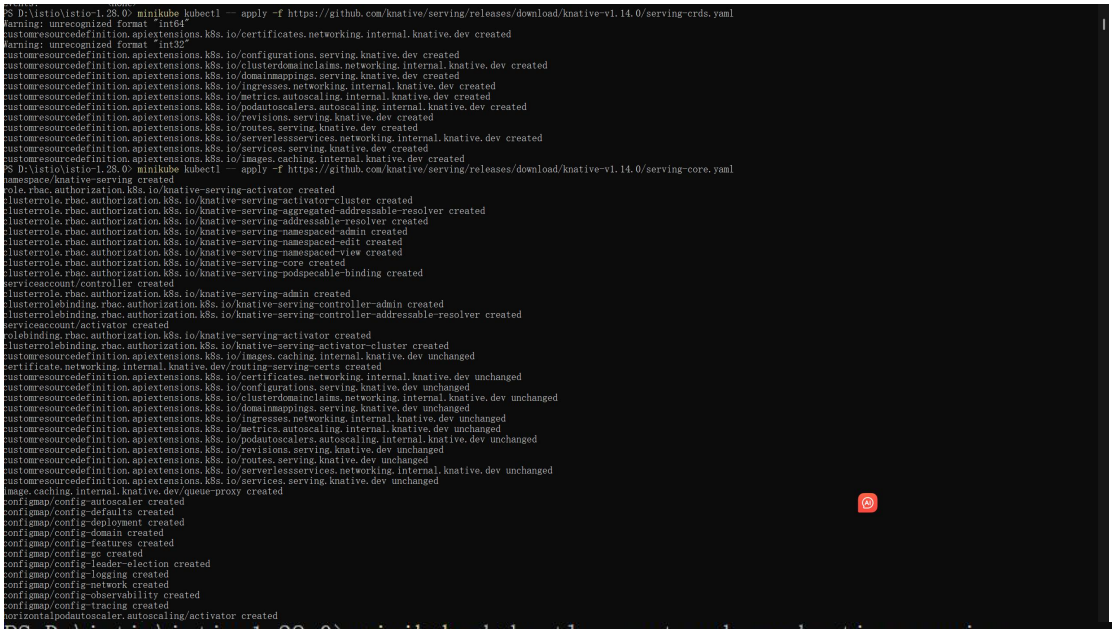
```yaml
apiVersion: serving.kserve.io/v1beta1
kind: InferenceService
metadata:
  name: sklearn-iris
  namespace: default
spec:
  predictor:
    sklearn:
      # 使用官方示例模型
      storageUri: "gs://seldon-models/sklearn/iris"
      resources:
        limits:
          cpu: 1
          memory: 2Gi

```

```
PS D:\istio\istio-1.28.0> minikube kubectl -- apply -f D:\istio\istio-1.28.0\kserve.yaml --validate=false
namespace/kserve created
Warning: unrecognized format "int32"
Warning: unrecognized format "int64"
customresourcedefinition.apiextensions.k8s.io/clusterservingruntimes.serving.kserve.io created
customresourcedefinition.apiextensions.k8s.io/clusterstoragecontainers.serving.kserve.io created
customresourcedefinition.apiextensions.k8s.io/inferencegraphs.serving.kserve.io created
customresourcedefinition.apiextensions.k8s.io/inferenceservices.serving.kserve.io created
customresourcedefinition.apiextensions.k8s.io/servingruntimes.serving.kserve.io created
customresourcedefinition.apiextensions.k8s.io/trainedmodels.serving.kserve.io created
serviceaccount/kserve-controller-manager created
role.rbac.authorization.k8s.io/kserve-leader-election-role created
clusterrole.rbac.authorization.k8s.io/kserve-manager-role created
clusterrole.rbac.authorization.k8s.io/kserve-proxy-role created
rolebinding.rbac.authorization.k8s.io/kserve-leader-election-rolebinding created
clusterrolebinding.rbac.authorization.k8s.io/kserve-manager-rolebinding created
clusterrolebinding.rbac.authorization.k8s.io/kserve-proxy-rolebinding created
configmap/inferenceservice-config created
secret/kserve-webhook-server-secret created
service/kserve-controller-manager-metrics-service created
service/kserve-controller-manager-service created
service/kserve-webhook-server-service created
deployment.apps/kserve-controller-manager created
certificate.cert-manager.io/serving-cert created
issuer.cert-manager.io/selfsigned-issuer created
mutatingwebhookconfiguration.admissionregistration.k8s.io/inferenceservice.serving.kserve.io created
validatingwebhookconfiguration.admissionregistration.k8s.io/clusterservingruntime.serving.kserve.io created
validatingwebhookconfiguration.admissionregistration.k8s.io/inferencegraph.serving.kserve.io created
validatingwebhookconfiguration.admissionregistration.k8s.io/inferenceservice.serving.kserve.io created
validatingwebhookconfiguration.admissionregistration.k8s.io/servingruntime.serving.kserve.io created
validatingwebhookconfiguration.admissionregistration.k8s.io/trainedmodel.serving.kserve.io created
PS D:\istio\istio-1.28.0>
```




```
PS D:\istio\istio-1.28.0> minikube kubectl -- get crds | findstr inferenceservice
inferenceservices.serving.kserve.io           2025-12-23T07:26:11Z
PS D:\istio\istio-1.28.0> minikube kubectl -- get crds | findstr inferenceservices
inferenceservices.serving.kserve.io           2025-12-23T07:26:11Z
PS D:\istio\istio-1.28.0>
PS D:\istio\istio-1.28.0> minikube kubectl -- get pods -n kserve
NAME                                         READY   STATUS    RESTARTS      AGE
kserve-controller-manager-6b7f685bc5-vll47   2/2     Running   1 (94s ago)   4m34s
PS D:\istio\istio-1.28.0> minikube kubectl -- api-resources | findstr inferenceservice
inferenceservices                 isvc         serving.kserve.io/v1beta1         true         InferenceService
PS D:\istio\istio-1.28.0>
```




```
PS D:\istio\istio-1.28.0> minikube kubectl -- get pods -n knative-serving
NAME                                READY   STATUS    RESTARTS   AGE
activator-dd5bc8c6f-nwfgv           1/1     Running   0          3m57s
autoscaler-76d567c78f-tc6fk         1/1     Running   0          3m57s
controller-7847f6b558-dd4q4         1/1     Running   0          3m57s
webhook-dc79f7ddb-rvghh             1/1     Running   0          3m56s
```

```
PS C:\WINDOWS\system32> minikube kubectl -- get pods -n kserve
NAME                                         READY   STATUS    RESTARTS      AGE
kserve-controller-manager-6b7f685bc5-vl147   2/2     Running   4 (27m ago)   38m
PS C:\WINDOWS\system32> cd D:\istio\istio-1.28.0
PS D:\istio\istio-1.28.0> minikube kubectl -- apply -f path/to/kserve-sklearnserver.yaml
error: the path "path/to/kserve-sklearnserver.yaml" does not exist
PS D:\istio\istio-1.28.0> minikube kubectl -- apply -f path/to/kserve-sklearnserver.yaml
error: the path "path/to/kserve-sklearnserver.yaml" does not exist
PS D:\istio\istio-1.28.0> minikube kubectl -- apply -f D:\istio\istio-1.28.0\runtimes\kserve-sklearnserver.yaml
clusterservingruntime.serving.kserve.io/kserve-sklearnserver created
PS D:\istio\istio-1.28.0> minikube kubectl -- get clusterservingruntime
NAME                  DISABLED   MODELTYPE   CONTAINERS         AGE
kserve-sklearnserver             sklearn     kserve-container   6s
PS D:\istio\istio-1.28.0> minikube kubectl -- delete inferenceservice sklearn-iris
inferenceservice.serving.kserve.io "sklearn-iris" deleted from default namespace
PS D:\istio\istio-1.28.0> minikube kubectl -- apply -f D:\istio\istio-1.28.0\sklearn-iris.yaml
inferenceservice.serving.kserve.io/sklearn-iris created
PS D:\istio\istio-1.28.0> minikube kubectl -- get pods -w
PS D:\istio\istio-1.28.0> minikube kubectl -- get clusterservingruntime
NAME                  DISABLED   MODELTYPE   CONTAINERS         AGE
kserve-sklearnserver             sklearn     kserve-container   4m40s
PS D:\istio\istio-1.28.0>
```

```
PS D:\istio\istio-1.28.0> minikube kubectl -- get pods -n knative-serving
NAME                                READY   STATUS    RESTARTS   AGE
activator-dd5bc8c6f-nwfgv           1/1     Running   0          36m
autoscaler-76d567c78f-tc6fk         1/1     Running   0          36m
controller-7847f6b558-dd4q4         1/1     Running   0          36m
webhook-dc79f7ddb-rvghh             1/1     Running   0          36m
```

```
PS D:\istio\istio-1.28.0> minikube kubectl -- apply -f D:\istio\istio-1.28.0\runtimes\kserve-sklearnserver.yaml
error: resource mapping not found for name: "kserve-sklearnserver" namespace: "" from "D:\\istio\\istio-1.28.0\\runtimes\\kserve-sklearnserver.yaml": no matches for kind "ClusterServingRuntime" in version "serving.kserve.io/v1beta1"
ensure CRDs are installed first
PS D:\istio\istio-1.28.0> minikube kubectl -- get crds | findstr kserve
clusterservingruntimes.serving.kserve.io          2025-12-23T07:26:11Z
clusterstoragecontainers.serving.kserve.io        2025-12-23T07:26:11Z
inferencegraphs.serving.kserve.io                 2025-12-23T07:26:11Z
inferenceservices.serving.kserve.io               2025-12-23T07:26:11Z
servingruntimes.serving.kserve.io                 2025-12-23T07:26:11Z
trainedmodels.serving.kserve.io                   2025-12-23T07:26:11Z
```

```
NAME           URL                                           READY   PREVIOUS   AGE
sklearn-iris   http://sklearn-iris.default.example.com       True               5m

NAME                                          READY   STATUS    RESTARTS   AGE
sklearn-iris-predictor-00001-deployment       1/1     Running   0          3m
```

```
{
  "predictions": [0]
}
```